

Phase 3 Structured Notes: Red-Black Tree Fundamentals and Insertion

0. Big Idea of Phase 3

Phase 3 teaches **Red-Black Trees**.

A **Red-Black Tree** is a **Binary Search Tree** that uses colors to stay balanced.

A normal Binary Search Tree, or BST, stores one key per node and follows this rule:

smaller values go left
larger values go right

A Red-Black Tree follows the same BST search rule, but each node also has a color:

RED or BLACK

The color helps control the shape of the tree so that it does not become a long chain.

Simple beginner idea:

A Red-Black Tree is a BST with extra color rules that keep it balanced.

1. Current Progress Before Phase 3

Before Phase 3, you already learned two important balanced tree ideas.

1.1 Phase 1: 2-3 Trees

In Phase 1, you learned about **2-3 trees**.

A 2-3 tree is a balanced multi-way search tree.

A 2-3 tree node can store:

1 key, or
2 keys

A 2-3 tree stays balanced because:

all leaves stay at the same level

You also learned:

- insertion happens at a leaf
- if a node gets too many keys, it splits
- if deletion creates an empty node, borrow first
- if borrowing is impossible, merge

1.2 Phase 2: 2-3-4 Trees

In Phase 2, you learned about **2-3-4 trees**.

A 2-3-4 tree is wider than a 2-3 tree.

A 2-3-4 tree node can store:

1 key,
2 keys, or
3 keys

A 2-3-4 tree also stays balanced because:

all leaves stay at the same level

The important connection is:

A Red-Black Tree is basically a 2-3-4 tree drawn in binary-tree form.

So your learning path is:

BST
↓
2-3 Tree
↓
2-3-4 Tree
↓
Red-Black Tree

1.3 Why Phase 3 comes after Phase 2

Red-Black Trees can look confusing if you only memorize rules.

But they become easier if you remember this:

Red-Black Trees use colors to represent the same balancing idea as 2-3-4 trees.

In a 2-3-4 tree, a node may contain multiple keys.

In a Red-Black Tree, every physical node contains only one key, but red nodes show which keys are mentally grouped together.

2. What Phase 3 Covers

Phase 3 covers these concepts:

1. What a Red-Black Tree is
 2. Red-Black Tree node structure
 3. The six Red-Black Tree rules
 4. Black depth
 5. Null pointers as black nodes
 6. Why Red-Black Trees are balanced
 7. Red-Black Tree insertion
 8. New nodes starting red
 9. Red-red conflict
 10. Parent, grandparent, and uncle
 11. Recoloring
 12. Rotations
 13. LL, RR, LR, and RL cases
 14. Insertion dry run
 15. Connection to 2-3-4 trees
 16. Common beginner mistakes
 17. Practice questions
 18. One-page memory sheet
-

3. What Is a Red-Black Tree?

A **Red-Black Tree** is a self-balancing Binary Search Tree.

It keeps the normal BST order:

```
left subtree < node < right subtree
```

But it adds colors:

```
RED  
BLACK
```

The colors are used to keep the tree approximately balanced.

3.1 Normal BST problem

A normal BST can become slow.

For example, if we insert sorted values:

```
10, 20, 30, 40, 50
```

A normal BST may become:

```
10  
  20  
    30  
      40  
        50
```

This is shaped like a linked list.

Searching may take:

```
O(n)
```

That is slow for large data.

3.2 Red-Black Tree solution

A Red-Black Tree avoids becoming a long chain by using color rules.

The tree is not perfectly balanced like an AVL tree, but it is balanced enough to guarantee fast operations.

Search, insert, and delete stay:

$O(\log n)$

4. Red-Black Tree Node Structure

A normal BST node stores:

```
data
left child
right child
```

A Red-Black Tree node stores:

```
data
color
left child
right child
parent pointer
```

4.1 C++ node structure

```
enum Color { RED, BLACK };

struct Node {
    int data;
    Color color;
    Node* left;
    Node* right;
    Node* parent;

    Node(int val) {
        data = val;
        color = RED;
        left = right = parent = nullptr;
    }
};
```

4.2 Meaning of each field

data

This stores the key value.

Example:

```
data = 40;
```

color

This stores the node color.

Possible values:

```
RED  
BLACK
```

left

This points to the left child.

The left child contains smaller values.

right

This points to the right child.

The right child contains larger values.

parent

This points to the parent node.

The parent pointer helps during insertion repair.

When fixing a Red-Black Tree, we often need to know:

```
Who is my parent?  
Who is my grandparent?  
Who is my uncle?
```

The parent pointer makes this easier.

`nullptr`

A pointer is `nullptr` when the child does not exist.

Example:

```
left = right = parent = nullptr;
```

This means the node currently has no left child, no right child, and no parent.

5. Why New Nodes Start Red

In the constructor, the new node is colored red:

```
color = RED;
```

This is very important.

A new Red-Black Tree node starts as **red**, not black.

5.1 Why not insert new nodes as black?

If we insert a new black node, one path gets an extra black node.

That may break the black-depth rule immediately.

Example idea:

```
Before insertion, all paths have black count 2.  
If we insert a black node on only one path, that path may now have black count  
3.
```

That is not allowed.

5.2 Why red is safer

A red node does not increase black depth.

So the tree first tries to insert the new node as red.

Then it only repairs the tree if a red-red conflict appears.

Simple memory:

New nodes start red because red does not increase black depth.

6. The Six Red-Black Tree Rules

A Red-Black Tree must follow six important rules.

Think of these rules as the safety rules that keep the tree balanced.

Rule	Name	Meaning	Simple Explanation
1	Color rule	Every node is red or black.	Each node has one of two colors.
2	Root rule	The root is always black.	The top node must be stable.
3	Null rule	Null pointers are black.	Missing children count as black nodes.
4	Black-depth rule	Every path from root to null has the same number of black nodes.	All paths must have equal black count.
5	No red-red rule	A red node cannot have a red parent or red child.	Red nodes cannot touch each other vertically.
6	New-node rule	A newly inserted node starts red.	Insert gently first, then repair if needed.

7. Rule 1: Every Node Is Red or Black

Every real node must have exactly one color:

RED or BLACK

Example:

20B means node 20 is black
30R means node 30 is red

The color is extra information used to control balance.

8. Rule 2: The Root Is Always Black

The root is the top node of the tree.

In a Red-Black Tree, the root must always be black.

Example:

```
20B
```

If insertion or recoloring makes the root red, we fix it at the end:

```
root->color = BLACK;
```

Simple memory:

Always force the root to black after insertion repair.

9. Rule 3: Null Pointers Are Black

In a Red-Black Tree, missing children are treated as black nodes.

Example:

```
  20B
 /
null null
```

Both `null` children count as black.

This matters when checking black depth.

Beginner warning:

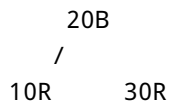
Null children count as black. Do not ignore them.

10. Rule 4: Same Black Depth on Every Path

Black depth means the number of black nodes on a path from the root to a null child.

Every path must have the same black count.

10.1 Example



Now trace the paths.

Path 1:

20B -> 10R -> null

Black count:

20B = 1
10R = still 1
null = 2

Path 2:

20B -> 30R -> null

Black count:

20B = 1
30R = still 1
null = 2

Both paths have black count 2.

So the black-depth rule is satisfied.

10.2 Red nodes do not increase black depth

If a node is red, we do not add to the black count.

Example:

```
20B -> 10R -> null
```

Only `20B` and `null` count as black.

11. Rule 5: No Red-Red Parent and Child

A red node cannot have a red parent.

A red node also cannot have a red child.

Invalid example:

```
20B
  30R
    40R
```

This is invalid because:

```
30R and 40R touch
```

That is called a **red-red conflict**.

Valid example:

```
  30B
 /
20R  40R
```

This is valid because both red nodes have a black parent.

Simple memory:

Red is allowed. Red-red is not allowed.

12. Rule 6: New Nodes Start Red

When inserting into a Red-Black Tree, the new node is first colored red.

Example:

```
insert 40
```

New node:

```
40R
```

Then we check whether the tree rules are broken.

If the parent is black, the insertion is fine.

If the parent is red, we must repair the tree.

13. Black Depth in Simple Words

Black depth is one of the most important Red-Black Tree ideas.

It means:

Count black nodes from the root down to a null child.

Every path must have the same black count.

13.1 What counts?

Counts as black:

```
black nodes  
null children
```

Does not count as black:

```
red nodes
```

13.2 Why black depth matters

If every path has the same black count, the tree cannot become extremely unbalanced.

A path can have red nodes, but red nodes cannot appear back-to-back.

So the tree height is still controlled.

This is why Red-Black Trees guarantee:

$$O(\log n)$$

for search, insertion, and deletion.

14. Red-Black Trees Compared with AVL Trees

Both AVL trees and Red-Black Trees are balanced BSTs.

But they balance in different ways.

14.1 AVL tree

An AVL tree is very strict about height balance.

It keeps the left and right subtree heights very close.

Because of this, AVL trees may rotate more often.

14.2 Red-Black Tree

A Red-Black Tree is less strict.

It allows the tree to be slightly taller than an AVL tree.

But it still guarantees logarithmic height.

Simple comparison:

Tree Type	Balance Style	Rotations
AVL Tree	Very strict	May rotate more often
Red-Black Tree	More relaxed	Often fewer rotations

Beginner idea:

Red-Black Trees trade perfect balance for fewer repairs while still staying fast.

15. Red-Black Tree Insertion: Big Picture

Insertion has two main stages.

Stage 1: Insert like a normal BST

Use the BST rule:

```
smaller values go left  
larger values go right
```

Find the correct null position.

Attach the new node there.

Color it red.

Stage 2: Fix Red-Black Tree rules

After insertion, check whether any rule is broken.

The most common problem is:

```
red parent + red child
```

This is called a red-red conflict.

To fix it, we use:

```
recoloring  
or  
rotation
```

16. Insertion Decision Table

After inserting a new red node, use this table.

Situation	What to Do
New node is root	Make it black
Parent is black	Done
Parent is red and uncle is red	Recolor
Parent is red and uncle is black or null	Rotate

This is the heart of Red-Black Tree insertion.

Simple memory:

```
Red uncle -> recolor
Black/null uncle -> rotate
```

17. Red-Red Conflict

A red-red conflict happens when a red node has a red parent.

Example:

```
20B
  30R
    40R
```

Here:

```
30R is red
40R is red
30R is the parent of 40R
```

This is illegal.

So we must repair the tree.

18. Parent, Grandparent, and Uncle

When a red-red conflict happens, we look at these nodes:

```
grandparent
|
parent
|
new node
```

We also look at the uncle.

18.1 Parent

The parent is the node directly above the new node.

18.2 Grandparent

The grandparent is the parent of the parent.

18.3 Uncle

The uncle is the sibling of the parent.

Example:

```
      G
     /
    P  U
   /
  N
```

Here:

```
G = grandparent
P = parent
U = uncle
N = newly inserted node
```

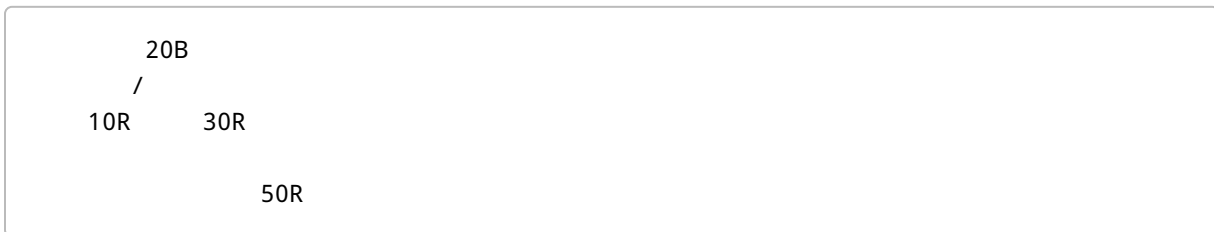
The uncle's color decides the repair case.

19. Case 1: Uncle Is Red → Recolor

If the parent is red and the uncle is also red, we recolor.

19.1 Example

Suppose we insert and get:



The new node is:

Its parent is:

The parent is red, so there is a red-red conflict.

Now find the uncle.

The grandparent is:

The uncle is:

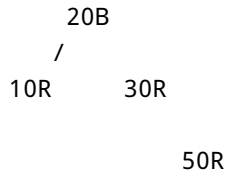
The uncle is red.

So we recolor.

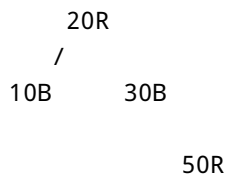
19.2 Recoloring steps

parent becomes BLACK
uncle becomes BLACK
grandparent becomes RED

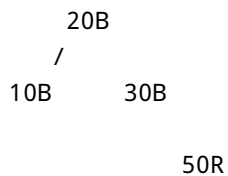
Before:



After recoloring:



But if the grandparent is the root, force it back to black:



19.3 Why recoloring works

Recoloring fixes the red-red conflict without changing the BST order.

It is similar to splitting a full 2-3-4 node and pushing a key upward.

Simple memory:

Red uncle means recolor.

20. Case 2: Uncle Is Black or Null → Rotate

If the parent is red but the uncle is black or null, recoloring alone is not enough.

We must rotate.

Simple memory:

Black or null uncle means rotate.

There are four possible shapes:

LL
RR
LR
RL

These shapes describe where the new node is compared to its parent and grandparent.

21. Rotation Meaning

A rotation is a small local rearrangement of the tree.

It changes the shape of the tree but keeps the BST order correct.

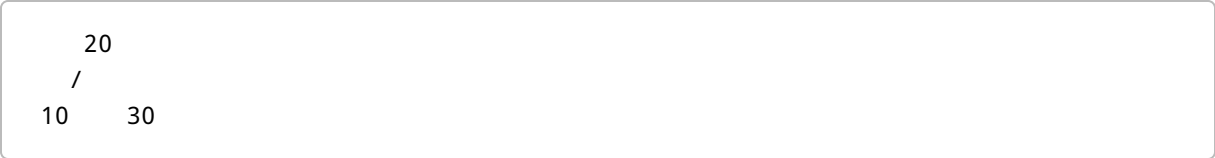
Example values:

10 < 20 < 30

Before rotation:

10
 20
 30

After rotation:



The order is still correct:

10 < 20 < 30

So rotation does not break the search rule.

22. The Four Rotation Cases

Case	Shape	Repair
LL	New node is left child of a left parent	Right rotate grandparent
RR	New node is right child of a right parent	Left rotate grandparent
LR	New node is right child of a left parent	Left rotate parent, then right rotate grandparent
RL	New node is left child of a right parent	Right rotate parent, then left rotate grandparent

22.1 Straight-line cases

These are straight-line cases:

LL
RR

They need one rotation.

22.2 Zig-zag cases

These are zig-zag cases:

LR
RL

They need two rotations.

Simple memory:

Straight line = one rotation. Zig-zag = two rotations.

23. LL Case

LL means:

new node is left child of a left parent

Shape:

```
      G
     /
    P
   /
  N
```

Repair:

right rotate around grandparent

Example:

```
      30B
     /
    20R
   /
  10R
```

This is red-red because 20R and 10R touch.

Uncle is black or null.

So rotate right around 30 .

Result idea:

```
      20B
     /
    10R   30R
```

24. RR Case

RR means:

new node is right child of a right parent

Shape:

G
P
N

Repair:

left rotate around grandparent

Example:

10B
20R
30R

This is red-red because 20R and 30R touch.

Uncle is black or null.

So rotate left around 10.

Result:

20B
/
10R 30R

25. LR Case

LR means:

new node is right child of a left parent

Shape:

```
      G
     /
    P
   /
  N
```

This is a zig-zag.

Repair:

1. Left rotate parent
2. Right rotate grandparent

Example:

```
      30B
     /
    10R
   /
  20R
```

First, rotate left at 10 :

```
      30B
     /
    20R
   /
  10R
```

Now it becomes an LL shape.

Then rotate right at 30 :

```

      20B
     /
    10R   30R

```

26. RL Case

RL means:

new node is left child of a right parent

Shape:

```

  G
   /
  P
 /
N

```

This is also a zig-zag.

Repair:

1. Right rotate parent
2. Left rotate grandparent

Example:

```

30B
   /
  50R
 /
40R

```

First, rotate right at 50 :

```

30B
   /
  40R

```


50R

Now it becomes an RR shape.

Then rotate left at 30 :

```
      40B
     /
30R   50R
```

Simple memory:

RL means right-left, so first fix the right-left bend, then rotate the grandparent.

27. Conceptual Insertion Fix Function

The insertion repair idea can be described like this:

```
void fixViolation(Node* &root, Node* pt) {
    Node* parent_pt = nullptr;
    Node* grand_parent_pt = nullptr;

    while ((pt != root) &&
           (pt->color != BLACK) &&
           (pt->parent->color == RED)) {

        parent_pt = pt->parent;
        grand_parent_pt = pt->parent->parent;

        Node* uncle_pt;

        if (parent_pt == grand_parent_pt->left) {
            uncle_pt = grand_parent_pt->right;
        } else {
            uncle_pt = grand_parent_pt->left;
        }

        if (uncle_pt != nullptr && uncle_pt->color == RED) {
            grand_parent_pt->color = RED;
            parent_pt->color = BLACK;
            uncle_pt->color = BLACK;
            pt = grand_parent_pt;
        }
    }
}
```

```
        } else {  
            // Rotate depending on LL, RR, LR, or RL case.  
        }  
    }  
  
    root->color = BLACK;  
}
```

27.1 What this function means in simple words

The loop continues while there is a red-red conflict.

It checks:

```
Is the current node red?  
Is the parent red?
```

If yes, it finds the grandparent and uncle.

Then:

```
red uncle -> recolor  
black/null uncle -> rotate
```

At the end:

```
root->color = BLACK;
```

This protects the root rule.

28. Full Red-Black Insertion Algorithm

Here is the beginner version of the algorithm.

Step 1: Insert using BST logic

Find where the key belongs.

```
smaller -> left  
larger -> right
```

Step 2: Color the new node red

```
new node = RED
```

Step 3: If the new node is root, make it black

```
root = BLACK
```

Step 4: If parent is black, stop

No Red-Black rule is broken.

Step 5: If parent is red, there is a red-red conflict

Now find:

```
parent  
grandparent  
uncle
```

Step 6: If uncle is red, recolor

```
parent -> BLACK  
uncle -> BLACK  
grandparent -> RED
```

Then continue checking upward.

Step 7: If uncle is black or null, rotate

Use the correct case:

```
LL, RR, LR, or RL
```

Step 8: Force root black

At the end:

root -> BLACK

29. Insertion Dry Run: 10, 20, 30

This is the easiest rotation example.

Step 1: Insert 10

The tree is empty.

Insert 10 as red first:

10R

But 10 is the root, so force it black:

10B

Step 2: Insert 20

20 is greater than 10, so it goes right.

10B

20R

The parent of 20 is black.

So no repair is needed.

Step 3: Insert 30

30 is greater than 10 and greater than 20.

It goes right of 20.

10B

20R

30R

Now there is a red-red conflict:

20R and 30R

The uncle is null, so it counts as black.

This is an RR case.

Repair:

left rotate around 10

After rotation and recoloring:

```
      20B
     /
10R  /  30R
```

Now the tree is valid.

30. Insertion Dry Run from Phase 3

The Phase 3 insertion sequence is:

10, 20, 30, 50, 40, 60, 70, 80, 4, 8

This sequence shows normal insertion, recoloring, rotations, and zig-zag cases.

30.1 Insert 10

10 becomes the root.

New nodes start red, but the root must be black.

10B

Rule shown:

root is black

30.2 Insert 20

20 is greater than 10, so it goes right.

10B

20R

Parent is black, so no conflict.

Rule shown:

red child with black parent is allowed

30.3 Insert 30

30 goes right of 20.

10B

20R

30R

This creates a red-red conflict.

Parent:

20R

Grandparent:

10B

Uncle:

null, treated as black

Shape:

RR

Repair:

left rotate around 10

Result:

```
      20B
     /
10R   30R
```

Rule shown:

black/null uncle -> rotate

30.4 Insert 50

50 goes right of 30.

Before repair:

```
      20B
     /
10R   30R
      50R
```

There is a red-red conflict:

30R and 50R

Parent:

30R

Grandparent:

20B

Uncle:

10R

The uncle is red.

Repair:

recolor

Parent 30 becomes black.

Uncle 10 becomes black.

Grandparent 20 becomes red, but it is the root, so force it black.

Result:

```
      20B
     /
10B  30B
     \
      50R
```

Rule shown:

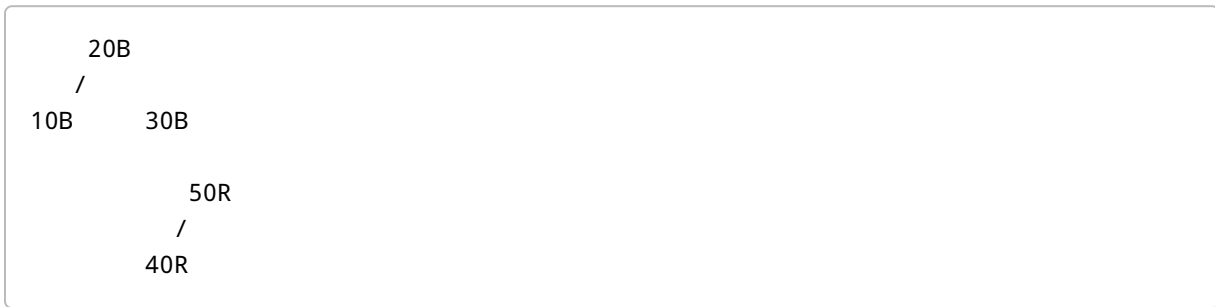
red uncle -> recolor

30.5 Insert 40

40 is greater than 20, greater than 30, but less than 50.

So it goes left of 50.

Before repair:



There is a red-red conflict:

50R and 40R

Parent:

50R

Grandparent:

30B

Uncle:

null, treated as black

Shape:

RL

Repair:

right rotate at 50
left rotate at 30

Result idea:

```
      20B
     /
10B   40B
     /
    30R   50R
```

Rule shown:

zig-zag case uses two rotations

30.6 Insert 60

60 goes right of 50.

Before repair:

```
      20B
     /
10B   40B
     /
    30R   50R
           60R
```

There is a red-red conflict:

50R and 60R

Parent:

50R

Grandparent:

40B

Uncle:

30R

The uncle is red.

Repair:

recolor

Parent 50 becomes black.

Uncle 30 becomes black.

Grandparent 40 becomes red.

The parent of 40 is 20, which is black, so stop.

Result idea:

```
      20B
     /
10B /   40R
    /
   30B   50B
        /
       60R
```

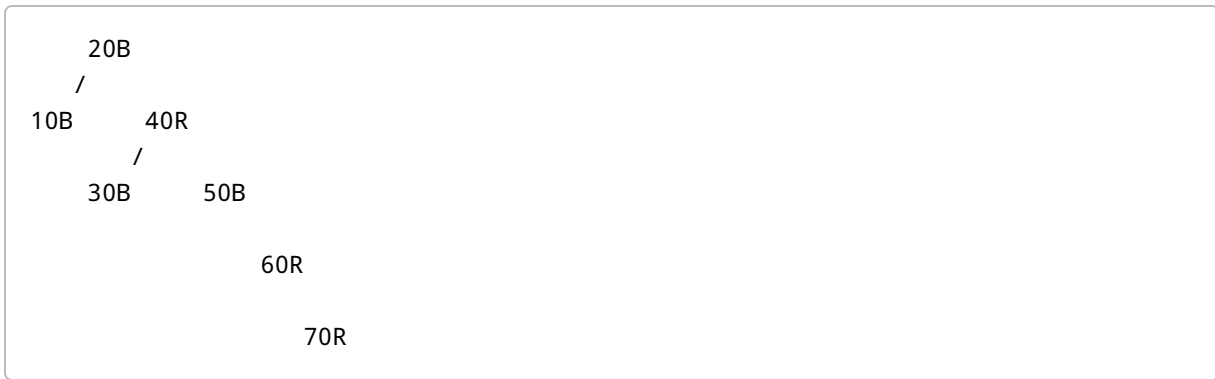
Rule shown:

recoloring can move the problem upward

30.7 Insert 70

70 goes right of 60.

Before repair:



There is a red-red conflict:

60R and 70R

Parent:

60R

Grandparent:

50B

Uncle:

null, treated as black

Shape:

RR

Repair:

left rotate around 50

Result idea:

20B
/



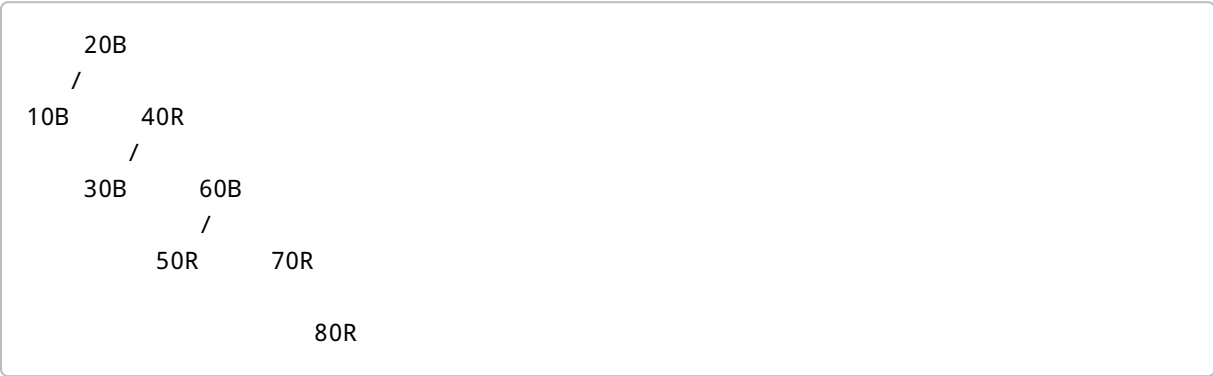
Rule shown:

RR case uses left rotation

30.8 Insert 80

80 goes right of 70.

Before first repair:



There is a red-red conflict:

70R and 80R

Parent:

70R

Grandparent:

60B

Uncle:

50R

The uncle is red.

So first repair:

recolor 70 and 50 black
make 60 red

Now 60 is red and its parent 40 is also red.

That creates a new red-red conflict higher up.

Now repair the higher conflict.

Parent:

40R

Grandparent:

20B

Uncle:

10B

Uncle is black.

So rotate.

Result idea:

```
      40B
     /
    20R  60R
   / \  /
  10B 30B 50B 70B
```

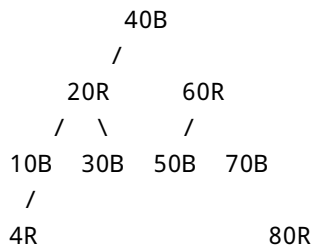
80R

Rule shown:

recoloring may cause a conflict higher up

30.9 Insert 4

4 goes left of 10.



Parent 10 is black.

No repair is needed.

Rule shown:

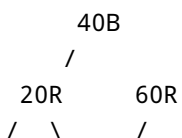
red node under black parent is fine

30.10 Insert 8

8 is less than 40, less than 20, less than 10, but greater than 4.

So it goes right of 4.

Before repair:



```

10B  30B  50B  70B
/
4R           80R

      8R

```

There is a red-red conflict:

4R and 8R

Parent:

4R

Grandparent:

10B

Uncle:

null, treated as black

Shape:

LR

Repair:

```

left rotate at 4
right rotate at 10

```

Result idea:

```

      40B
      /
    20R  60R
   /  \  /
  8B 30B 50B 70B

```


/ \
4R 10R 80R

Rule shown:

LR zig-zag case uses two rotations

31. How Red-Black Trees Connect to 2-3-4 Trees

This is the most important mental bridge.

A Red-Black Tree is a binary-tree version of a 2-3-4 tree.

31.1 Black node meaning

A black node represents a separate 2-3-4 tree node.

Example:

20B

This can represent a 2-3-4 node:

[20]

31.2 Red node meaning

A red node represents an extra key grouped with its black parent.

Example:

20B

40R

This can represent a 2-3-4 node:

[20, 40]

31.3 4-node representation

A 2-3-4 node with three keys:

[10, 20, 30]

can be drawn as:

```
      20B
     /
10R   30R
```

The black node is the middle key.

The red children are extra keys grouped with it.

31.4 Why this matters

Recoloring and rotations are not random.

They are the binary-tree way of doing the same balancing work as 2-3-4 trees.

In simple terms:

Red-Black recoloring is like 2-3-4 splitting.
Red-Black rotation is like rearranging grouped keys.

32. Recoloring and 2-3-4 Splitting

In a 2-3-4 tree, when a node becomes too full, it splits and pushes a key upward.

In a Red-Black Tree, when parent and uncle are red, recoloring does something similar.

Example Red-Black shape:

```
      20B
     /
10R   30R
```

This represents a 2-3-4 node:

[10, 20, 30]

If we insert another key below it, the structure may need to split.

Red-Black recoloring helps push the problem upward, just like a split in a 2-3-4 tree.

Simple memory:

Recoloring is the Red-Black version of pushing balance upward.

33. Rotations and 2-3-4 Rearrangement

Sometimes the keys are in a shape that does not correctly represent the 2-3-4 grouping.

Rotation fixes the binary shape while keeping the search order.

Example:

```
10B
  20R
    30R
```

This is not a valid Red-Black shape because of red-red conflict.

After rotation:

```
    20B
   /
10R  30R
```

Now the tree represents the grouped keys cleanly:

[10, 20, 30]

Simple memory:

Rotation changes the shape, not the sorted order.

34. Complete Teaching Program for Phase 3

This program implements Red-Black Tree insertion with rotations and recoloring.

It is useful for learning how insertion repair works.

```
#include <iostream>
#include <iomanip>
using namespace std;

enum Color { RED, BLACK };

struct Node {
    int data;
    Color color;
    Node *left, *right, *parent;

    explicit Node(int value)
        : data(value), color(RED), left(nullptr), right(nullptr),
        parent(nullptr) {}
};

class RedBlackTree {
private:
    Node* root = nullptr;

    string colorName(Color c) const {
        return c == RED ? "R" : "B";
    }

    void rotateLeft(Node* x) {
        Node* y = x->right;

        x->right = y->left;
        if (y->left != nullptr) {
            y->left->parent = x;
        }

        y->parent = x->parent;

        if (x->parent == nullptr) {
            root = y;
        } else if (x == x->parent->left) {
            x->parent->left = y;
        } else {
            x->parent->right = y;
        }
    }
};
```

```

    }

    y->left = x;
    x->parent = y;
}

void rotateRight(Node* x) {
    Node* y = x->left;

    x->left = y->right;
    if (y->right != nullptr) {
        y->right->parent = x;
    }

    y->parent = x->parent;

    if (x->parent == nullptr) {
        root = y;
    } else if (x == x->parent->right) {
        x->parent->right = y;
    } else {
        x->parent->left = y;
    }

    y->right = x;
    x->parent = y;
}

void fixViolation(Node* pt) {
    while (pt != root && pt->parent->color == RED) {
        Node* parent = pt->parent;
        Node* grand = parent->parent;

        if (parent == grand->left) {
            Node* uncle = grand->right;

            if (uncle != nullptr && uncle->color == RED) {
                parent->color = BLACK;
                uncle->color = BLACK;
                grand->color = RED;
                pt = grand;
            } else {
                if (pt == parent->right) {
                    pt = parent;
                    rotateLeft(pt);
                    parent = pt->parent;
                    grand = parent->parent;
                }
            }
        }
    }
}

```

```

        parent->color = BLACK;
        grand->color = RED;
        rotateRight(grand);
    }
} else {
    Node* uncle = grand->left;

    if (uncle != nullptr && uncle->color == RED) {
        parent->color = BLACK;
        uncle->color = BLACK;
        grand->color = RED;
        pt = grand;
    } else {
        if (pt == parent->left) {
            pt = parent;
            rotateRight(pt);
            parent = pt->parent;
            grand = parent->parent;
        }

        parent->color = BLACK;
        grand->color = RED;
        rotateLeft(grand);
    }
}

root->color = BLACK;
}

void printSideways(Node* node, int depth) const {
    if (node == nullptr) return;

    printSideways(node->right, depth + 1);
    cout << setw(depth * 5) << "" << node->data << colorName(node->color)
<< "\n";
    printSideways(node->left, depth + 1);
}

public:
void insert(int key) {
    Node* pt = new Node(key);
    Node* parent = nullptr;
    Node* current = root;

    while (current != nullptr) {
        parent = current;

```

```

        if (key < current->data) {
            current = current->left;
        } else if (key > current->data) {
            current = current->right;
        } else {
            cout << "Duplicate ignored: " << key << "\n";
            delete pt;
            return;
        }
    }

    pt->parent = parent;

    if (parent == nullptr) {
        root = pt;
    } else if (key < parent->data) {
        parent->left = pt;
    } else {
        parent->right = pt;
    }

    fixViolation(pt);
}

void print() const {
    cout << "Tree printed sideways (R=red, B=black):\n";
    printSideways(root, 0);
    cout << "\n";
}

};

int main() {
    RedBlackTree tree;

    int keys[] = {10, 20, 30, 50, 40, 60, 70, 80, 4, 8};

    for (int key : keys) {
        cout << "Insert " << key << "\n";
        tree.insert(key);
        tree.print();
    }

    return 0;
}

```

35. What the Program Teaches

35.1 `enum Color`

```
enum Color { RED, BLACK };
```

This creates two possible color values.

35.2 `Node`

Each node stores:

```
data  
color  
left pointer  
right pointer  
parent pointer
```

35.3 `rotateLeft`

This fixes RR-type shapes.

Example:

```
10  
  20  
    30
```

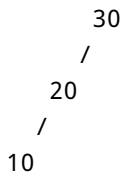
becomes:

```
  20  
 /  
10  30
```

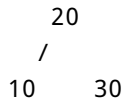
35.4 `rotateRight`

This fixes LL-type shapes.

Example:



becomes:



35.5 fixViolation

This repairs red-red conflicts.

It checks:

```

Is the parent red?
What color is the uncle?
Which shape is it: LL, RR, LR, or RL?

```

Then it chooses recoloring or rotation.

35.6 insert

This first inserts the key like a normal BST.

Then it calls:

```
fixViolation(pt);
```

35.7 printSideways

This prints the tree sideways.

The right subtree appears above the node.

The left subtree appears below the node.

Example:

```

      30R
    20B
      10R

```

This means:

```

      20B
     /
    10R  30R

```

36. Complexity

Because Red-Black Trees stay balanced, operations are efficient.

Operation	Time Complexity	Simple Meaning
Search	$O(\log n)$	We travel down the height of the tree.
Insert	$O(\log n)$	We search down, then repair upward if needed.
Delete	$O(\log n)$	Deletion is also bounded by tree height.
Storage	$O(n)$	We store all nodes.

36.1 Why insertion is $O(\log n)$

Insertion has two parts:

1. Search down to find the correct position.
2. Repair using recoloring or rotations.

The tree height is logarithmic, so the operation is:

```

O(log n)

```

Rotations themselves are local operations.

A single rotation takes:

```

O(1)

```

The logarithmic cost comes from moving down or up the tree height.

37. Common Beginner Mistakes

Mistake 1: Thinking red means wrong

Wrong idea:

A red node is always bad.

Correct idea:

A red node is allowed.
A red node with a red parent is bad.

Mistake 2: Forgetting null pointers are black

Null children count as black.

This matters when checking black depth and uncle color.

Mistake 3: Inserting new nodes as black

New nodes should start red.

If they start black, black depth may break immediately.

Mistake 4: Forgetting to force the root black

After recoloring, the root might become red.

Always finish by making the root black.

Mistake 5: Checking only the new node

The main insertion problem is not just the new node.

The problem is:

new node is red AND parent is red

Mistake 6: Confusing uncle with sibling

The uncle is not the sibling of the new node.

The uncle is the sibling of the parent.

Mistake 7: Rotating when the uncle is red

If the uncle is red, recolor first.

Do not rotate immediately.

Mistake 8: Recoloring when the uncle is black or null

If the uncle is black or null, use rotation.

Mistake 9: Mixing up LL, RR, LR, and RL

Remember:

```
LL and RR are straight-line cases.  
LR and RL are zig-zag cases.
```

Mistake 10: Losing parent pointers during rotation

In real code, rotation must update:

```
child pointers  
parent pointers  
root pointer if needed
```

If parent pointers are not updated correctly, the tree structure breaks.

38. Key Vocabulary

Red-Black Tree

A balanced Binary Search Tree that uses red and black colors to control height.

Color

Extra information stored in each node.

Possible values:

RED
BLACK

Root

The top node of the tree.

In a Red-Black Tree, the root must be black.

Null pointer

A missing child.

In Red-Black Trees, null pointers are treated as black.

Black depth

The number of black nodes on a path from the root to a null child.

Red-red conflict

A violation where a red node has a red parent.

Parent

The node directly above another node.

Grandparent

The parent of the parent.

Uncle

The sibling of the parent.

Recoloring

Changing node colors to fix a red-red conflict.

Rotation

A local tree rearrangement that preserves BST order.

LL case

New node is left child of a left parent.

Repair with right rotation.

RR case

New node is right child of a right parent.

Repair with left rotation.

LR case

New node is right child of a left parent.

Repair with left rotation at parent, then right rotation at grandparent.

RL case

New node is left child of a right parent.

Repair with right rotation at parent, then left rotation at grandparent.

39. Phase 3 Final Summary

A Red-Black Tree is a balanced Binary Search Tree.

It follows normal BST ordering:

```
left < node < right
```

But it adds colors:

```
RED  
BLACK
```

The main Red-Black rules are:

1. Every node is red or black.
2. The root is black.
3. Null pointers are black.
4. Every root-to-null path has the same number of black nodes.
5. A red node cannot have a red parent or red child.
6. New nodes start red.

Insertion works like this:

1. Insert like a normal BST.
2. Color the new node red.
3. If the new node is root, make it black.
4. If the parent is black, stop.
5. If the parent is red, there is a red-red conflict.
6. Check the uncle.
7. Red uncle -> recolor.
8. Black/null uncle -> rotate.
9. Force the root black at the end.

The most important memory rule is:

Red uncle -> recolor
Black/null uncle -> rotate

Another important memory rule:

Straight line -> one rotation
Zig-zag -> two rotations

Red-Black Trees connect to 2-3-4 trees because red nodes act like extra keys grouped with black parent nodes.

Simple final sentence:

Red-Black insertion is normal BST insertion followed by fixing red-red conflicts using recoloring or rotations.

40. Mini Practice Questions

Question 1

What is a Red-Black Tree?

Answer:

A Red-Black Tree is a Binary Search Tree that uses red and black colors to stay balanced.

Question 2

What color is a newly inserted node?

Answer:

Red.

Reason:

A red node does not increase black depth.

Question 3

What color must the root always be?

Answer:

Black.

Question 4

Do null children count as red or black?

Answer:

Black.

Question 5

What is a red-red conflict?

Answer:

A red-red conflict happens when a red node has a red parent.

Question 6

If the parent is red and the uncle is red, what should we do?

Answer:

Recolor.

Question 7

If the parent is red and the uncle is black or null, what should we do?

Answer:

Rotate.

Question 8

What is the uncle?

Answer:

The uncle is the sibling of the parent.

Question 9

What does RR mean?

Answer:

The new node is the right child of a right parent.

Repair with a left rotation around the grandparent.

Question 10

What does LL mean?

Answer:

The new node is the left child of a left parent.

Repair with a right rotation around the grandparent.

Question 11

What does LR mean?

Answer:

The new node is the right child of a left parent.

Repair with left rotation at parent, then right rotation at grandparent.

Question 12

What does RL mean?

Answer:

The new node is the left child of a right parent.

Repair with right rotation at parent, then left rotation at grandparent.

Question 13

Why do rotations not break BST order?

Answer:

Because rotations rearrange nodes locally while keeping the sorted order the same.

Question 14

How are Red-Black Trees connected to 2-3-4 trees?

Answer:

A Red-Black Tree is a binary-tree version of a 2-3-4 tree. Red nodes represent extra keys grouped with black parent nodes.

Question 15

What is the time complexity of Red-Black Tree insertion?

Answer:

$O(\log n)$

41. One-Page Memory Sheet

Red-Black Tree = balanced BST with colors

Node has:

- data
- color
- left pointer
- right pointer
- parent pointer

Colors:

- RED
- BLACK

Rules:

1. Every node is red or black.
2. Root is black.
3. Null pointers are black.
4. Every root-to-null path has same black count.
5. No red-red parent-child.
6. New nodes start red.

Insertion:

1. Insert like BST.
2. New node is red.
3. If root, make black.
4. If parent black, done.
5. If parent red, red-red conflict.
6. Find uncle.
7. Red uncle -> recolor.
8. Black/null uncle -> rotate.
9. Force root black.

Rotation cases:

- LL -> right rotate grandparent
- RR -> left rotate grandparent
- LR -> left rotate parent, then right rotate grandparent
- RL -> right rotate parent, then left rotate grandparent

Memory rules:

Red uncle -> recolor

Black/null uncle -> rotate

Straight line -> one rotation

Zig-zag -> two rotations

Connection to 2-3-4 trees:

Black node = separate 2-3-4 node

Red node = extra key grouped with black parent

Complexity:

Search = $O(\log n)$

Insert = $O(\log n)$

Delete = $O(\log n)$

Storage = $O(n)$

42. Final One-Sentence Summary

A Red-Black Tree is a balanced BST where new nodes start red, red-red conflicts are fixed by recoloring or rotations, and the color rules keep all operations fast at $O(\log n)$.

End of Phase 3 Notes.